

UNIVERSITY OF LEEDS

COMP1850 Programming

Week 5 Session 2

Object Oriented Programming & Classes

Recap

UNIVERSITY OF LEEDS

Last time, we

- Learned how systems can be viewed as collections of **objects**, and how objects fall into categories called **classes**
- Noted that a class specifies a set of attributes and behaviours that characterise objects of that class
- Saw that software classes are **abstractions** of real-world classes, implementing attributes & behaviour as **fields** & **methods**
- Learned the basic Python syntax for creating classes

Today's Aims

UNIVERSITY OF LEEDS

- Investigate how we can make classes more robust
- Learn about the special 'dunder methods' that Python classes have, and how they can make our classes feel more like built-in types
- Become more comfortable with writing classes in Python

Reminder: `Point` Class

UNIVERSITY OF LEEDS

```
class Point:
    """
    A point in 2D space.
    """
    def __init__(self, x, y):
        """
        Creates a Point, given x & y coordinates.
        """
        self.x = x
        self.y = y

    def distance(self):
        """
        Returns distance of this Point from (0, 0).
        """
        return math.hypot(self.x, self.y)
```

Types of x and y?

Should we document this?

Can we force use of a particular type?

Type Hinting

UNIVERSITY OF LEEDS

- Python allow us to declare the types of values we expect to be passed as arguments to a function or method
- … and the type of value we expect to see returned
- **These are hints only**; there is no run-time type checking actually done by the Python VM!
- **Type hints are entirely optional**, but they can help to document code, and IDEs or tools such as mypy can use the information

Example

UNIVERSITY OF LEEDS

```
class Point:
    ...
    def __init__(self, x: float, y: float):
        """
        Creates a Point, given x & y coordinates.
        """
        self.x = x
        self.y = y

    def distance(self) -> float:
        """
        Returns distance of this Point from (0, 0).
        """
        return math.hypot(self.x, self.y)
```

Type Checking

UNIVERSITY OF LEEDS

- Python uses **dynamic typing**
- This means you cannot restrict the types of values that are used as arguments when a function or method is called
- ...but you can add code to a function or method that checks whether those types are appropriate, before doing anything else
- Use built-in function `isinstance()` with value and type(s) as arguments, and raise a `TypeError` if it returns `False`

```
if not isinstance(name, str):
    raise TypeError("name should be a string")
```

Example: `Point` Constructor

UNIVERSITY OF LEEDS

```
class Point:
    ...
    def __init__(self, x, y):
        """
        Creates a Point, given x & y coordinates.
        """
        if not isinstance(x, (int, float)):
            raise TypeError("x must be a number")

        if not isinstance(y, (int, float)):
            raise TypeError("y must be a number")

        self.x = x
        self.y = y
```

Note use of a tuple of allowed types here

Task 1

UNIVERSITY OF LEEDS

`week1.5/session2/task1/`

- Experiment with type hinting in `hinting.py`
- Then experiment with type checking in `checking.py`
- See `README.md` for detailed instructions

Comparison With Other Languages

UNIVERSITY OF LEEDS

- In dynamically-typed languages like Python, we don't know until run time whether argument types are acceptable
- In statically-typed languages like C or Kotlin, misuse of types is detected earlier, at compile time
- Python gives us freedom & flexibility, but with drawbacks
  - Explicit type checking is needed if we want robustness
  - ...but having type checking everywhere clutters code, so often isn't done
  - Instead, we tend to be very reliant on having good automated tests, to verify that code is being used correctly

Setter Methods

UNIVERSITY OF LEEDS

- The previous example only checks x & y in the constructor, but what if we are updating a point's x & y coordinates?
- A better solution might be to do the type checking and assignment of values to fields in separate 'setter methods'
- Setters can be called by class users to update a `Point` object's coordinates, and can also be called from within the constructor to create the fields representing those coordinates
- Note: when a method of a class calls another method of a class, **you need to use the '`self.`' prefix**, just as you do with fields

Example

UNIVERSITY OF LEEDS

```
class Point:
    ...
    def __init__(self, x, y):
        """
        Creates a Point, given x & y coordinates.
        """
        self.set_x(x)
        self.set_y(y)

    def set_x(self, x):
        """
        Updates x coordinate of this Point.
        """
        if not isinstance(x, (int, float)):
            raise TypeError("x must be a number")
        self.x = x
    ...
```

Things to Note

- We cannot enforce the use of `set_x()` and `set_y()`
- A user of `Point` is still free to assign directly to fields `x` and `y`, potentially giving them nonsensical values
- There are partial solutions (e.g., using **properties**), but Python is quite weak in this respect; other languages offer firmer guarantees that objects will never be in an invalid state

Dunder Methods

UNIVERSITY OF LEEDS

- You've used one of these already: `__init__()`
- There are many others that you can define in Python classes
- Their names all begin and end with double underscores
- ...hence they are often called 'dunder methods'
- Common feature is that **we don't invoke them using their names!**
- Instead, they are called for us in particular situations...

Overriding __str__()

UNIVERSITY OF LEEDS

- If you implement `__str__()` in a class, that method will be called automatically in contexts where you have provided an instance of the class but a string is expected
- For example, `print()` expects to be given strings; if you pass it something that isn't a string, that thing's `__str__()` method will be called to generate a suitable string
- Note: classes already have a (not very useful) version of `__str__()`; replacing this with your own version is called **overriding**
  - More on this next week...

Example

UNIVERSITY OF LEEDS

```
class Point:
    ...
    def __init__(self, x, y):
        """
        Creates a Point, given x & y coordinates.
        """
        self.x = x
        self.y = y

    def __str__(self):
        """
        Returns string representation of this Point.
        """
        return f"({self.x},{self.y})"
```

__str__() has no parameters (other than self)

__str__() must return a string

Task 2

UNIVERSITY OF LEEDS

`week1.5/session2/task2/`

- Edit `geometry.py` and implement `__str__()` in the `Point` and `Circle` classes
- In `print.py`, write code to create and print `Point` & `Circle` objects, to demonstrate use of these methods

__add__()

UNIVERSITY OF LEEDS

- Dunder method `__add__()` specifies what happens when you use the + operator with two instances of the class
- Example: you can implement it for a `Money` class, so that familiar syntax can be used to add together two sums of money

```
m1 = Money(1, 99)
m2 = Money(0, 50)

sum = m1 + m2

print(sum)          # prints "£2.49"
```

__add__()

UNIVERSITY OF LEEDS

`self` refers to the object on the left of the + operator

`other` refers to the object on the right of the + operator

```
def __add__(self, other):
    ...
```

Method body must return an instance of the class

Task 3

UNIVERSITY OF LEEDS

`week1.5/session2/task3/`

- Edit `money.py` and finish implementing the `Money` class, following the instructions in the README
  - You will need to implement `__str__()` and `__add__()`
- Edit `add.py` and write a program that creates two `Money` objects, prints them, adds them, then prints the result of the addition

__contains__()

UNIVERSITY OF LEEDS

- Dunder method `__contains__()` specifies what happens when you use the `in` operator with an instance of the class
- Example: you could implement this for the `Circle` class, and use it to test whether a point falls inside a circle

```
point = Point(1.0, 1.5)
circle = Circle(2.5)

if point in circle:
    print("point is inside circle!")
```

Task 4

UNIVERSITY OF LEEDS

`week1.5/session2/task4/`

- Edit `geometry.py` and implement `__contains__()` in the `Circle` class, following the guidance in the README
- Edit `inside.py` and write a program that creates `Circle` & `Point` objects, then tests whether the point is inside the circle

Making Classes Feel Like Collections

UNIVERSITY OF LEEDS

Provide implementations of

- `__getitem__()`, to support look-up via `[]` and an index
- `__setitem__()`, to support assignment via `[]` and an index
- `__contains__()`, to support membership testing
- `__len__()`, to support size queries with the `len()` function

A by-product of implementing `__getitem__()` is that instances of your class can be iterated over with `for` loops!

(You can also implement `__iter__()` to support iteration)

Task 5

UNIVERSITY OF LEEDS

`week1.5/session2/task5/`

- Edit `data.py` and finished implementing the `Dataset` class
- Write a program in `datademo.py` that creates a `Dataset` object from data in a file and then demonstrates the features of the class

Advanced Topics: Properties

UNIVERSITY OF LEEDS

- Python allows us to define properties, with associated backing fields, getter and (optionally) setter methods
- Accessing a property looks like accessing a field, but in reality the getter or setter method is being called
- The setter method of a property can do type & value checking, rejecting the assignment of a value if it is invalid
- You can also have read-only properties

https://python-basics-tutorial.readthedocs.io/en/latest/oop/property.html

Advanced Topics: Slots

UNIVERSITY OF LEEDS

- Normally, Python objects store fields as keys and values in a dictionary
- A more efficient approach, memory-wise, is to store them as **slots**
- A side-benefit is that slots are pre-defined by the class, which prevents users from introducing additional fields into an object

```
class Point:
    __slots__ = ("x", "y")
    ...
```

Summary

UNIVERSITY OF LEEDS

We have

- Considered how type hinting and type checking can be used to document classes better and make them more robust
- Looked at how we can write setter methods that validate attempts to modify the value of a field (though we cannot enforce use)
- Explored the use of dunder methods to customize the ways that instance of a class behave in different situations
- Given you more experience of working with classes in Python